CivicPulse

FYP Code: 23F-90

Software Design Specification

BY

AYOUSH KUMAR, CMS. ID: 023-23-0220 SARICHAND, CMS. ID: 023-23-0024

RAINUKA BAI, CMS. ID: 023-23-0053



Supervised by: Dr. Saif Hassan _____

Co-Supervised by: Dr. Ismail _____

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

NOVEMBER 2026

Document Revision History

Version	Date	Prepared by	Description
1.0	November 2026	Ayoush Kumar, Sarichand, Rainuka Bai	Initial version for supervisor review

Table of Contents

1. INTRODUCTION 1
   1.1. Purpose of Document 1
   1.2. Definitions, acronyms, and abbreviations 1
2. OVERALL SYSTEM DESCRIPTION 2
   2.1. Project Background 2
   2.2. Project Objectives 2
3. SYSTEM ARCHITECTURE 3
4. DOMAIN MODEL 5
5. CLASS DIAGRAM 6
6. DATABASE DIAGRAM 9
7. ENTITY RELATIONSHIP DIAGRAM (ERD) 13
8. SEQUENCE DIAGRAM 15
9. SYSTEM INTERFACE DESIGN 18
10. TEST CASES 20
11. REFERENCES 22

List of Figures

Figure 1: Detailed system architecture....3
Figure 2: Domain model....5
Figure 3: Class diagram, part 1: accounts, reports, merging and vision....6
Figure 4: Class diagram, part 2: ranking, weather, repair checks and rewards....7
Figure 5: Database diagram: accounts and rate limiting....9
Figure 6: Database diagram: reports, pins, images and analysis....10
Figure 7: Database diagram: missions, repair claims, points and coupons....11
Figure 8: Database diagram: weather, ranking settings and audit log....12
Figure 9: Entity relationship diagram: users, reports, pins and images....13
Figure 10: Entity relationship diagram: missions, rewards and weather....14
Figure 11: Sequence diagram: registration, verification and login....15
Figure 12: Sequence diagram: report submission and merging....16
Figure 13: Sequence diagram: repair verification and approval....17
Figure 14: Citizen application screen designs....18
Figure 15: Administrator dashboard screen designs....19

List of Tables

Table 1: Definitions, acronyms and abbreviations....1
Table 2: Project objectives and responsible design elements....2
Table 3: Principal architectural decisions....3
Table 4: API route groups....4
Table 5: Responsibilities of the service classes....8
Table 6: Principal constraints and indexes....12
Table 7: Citizen application screens....18
Table 8: Planned test cases....20

1. Introduction

1.1. Purpose of Document

This Software Design Specification describes how CivicPulse will satisfy the requirements in its Software Requirements Specification and approved proposal [12]. It records the architecture, domain concepts, classes, database, principal interactions, interface designs and planned tests. It is the common reference for developers, and it lets the evaluation committee examine the design behind each graded result. Components not yet implemented are described as planned work.

1.2. Definitions, acronyms, and abbreviations

Table 1: Definitions, acronyms and abbreviations

Term	Definition
Active window	The period after which an open pin no longer accepts new reports
API / REST	Application Programming Interface / Representational State Transfer
Award key	A unique value derived from user, reason and reference, which prevents a repeated award
DBSCAN	Density-Based Spatial Clustering of Applications with Noise, used in the offline radius sweep [7]
ERD	Entity Relationship Diagram
Gate	One of five sequential repair checks
JWT	JSON Web Token
ORB / RANSAC	Keypoint matching method [8] / method for rejecting inconsistent matches
ORM	Object-Relational Mapper, here SQLAlchemy
Pair threshold	The merge distance calculated for one report and one pin from their accuracy values
pHash / SHA-256	Perceptual image hash / cryptographic file hash
Pin	A single physical hazard on the map, to which reports are attached
Report	One citizen submission with photograph, category and position
YOLO	The object detection model family used for defects [9]

2. Overall System Description

2.1. Project Background

Every monsoon severely damages the road network of Sindh, and Sukkur Division alone recorded 1,055 km of damaged road in the 2022 floods [1]. Funding reaches only selected roads [2], so authorities must set priorities, although existing complaint systems provide limited support for doing so. One national portal reports near-total closure of complaints while many citizens consider them unresolved [3], and report-only platforms handle duplicates through staff effort or by displaying existing reports [4]. As a result, duplicate tickets obscure how many people are affected, severity is not measured and ticket closure gives no evidence of repair.

Citizens use the CivicPulse Android application to photograph hazards. The server groups duplicate reports into pins, estimates severity from the photograph and ranks open hazards for a reviewer. A repair is accepted only after its evidence has passed automatic checks and a human reviewer. Because a phone cannot be fully trusted, the server makes every consequential decision.

2.2. Project Objectives

Table 2: Project objectives and responsible design elements

ID	Objective	Responsible design elements
O1	One pin per physical hazard, measured against labelled data	Merge engine, PostGIS indexes, split operation, sweep script
O2	Defect detection and severity estimation from a photograph	Vision service, job worker, severity rule
O3	Verification of claimed repairs and measurement of false acceptance	Repair gate controller, ORB matcher, gate switches
O4	Hazard ranking with a transparent score	Priority engine, ranking settings, breakdown display
O5	Warnings to affected zones and raised priorities	Weather scheduler, push delivery, notification log
O6	Demonstrably correct points, ranking and coupons	Points ledger, unique keys, atomic redemption
O7 (stretch)	Observation of real use, subject to recruiting users	Existing logs; no additional design required

3. System Architecture

CivicPulse follows a three-tier client-server architecture, with background processes on the server. Both clients communicate only with the API. Time-consuming work runs outside the request path, so the phone does not wait for model inference. When the application uploads a report to report intake, the image service validates and stores the photograph, and the merge engine assigns the report to a pin in the database. A queued job then runs the vision service, and the priority engine updates the pin's score for the dashboard.

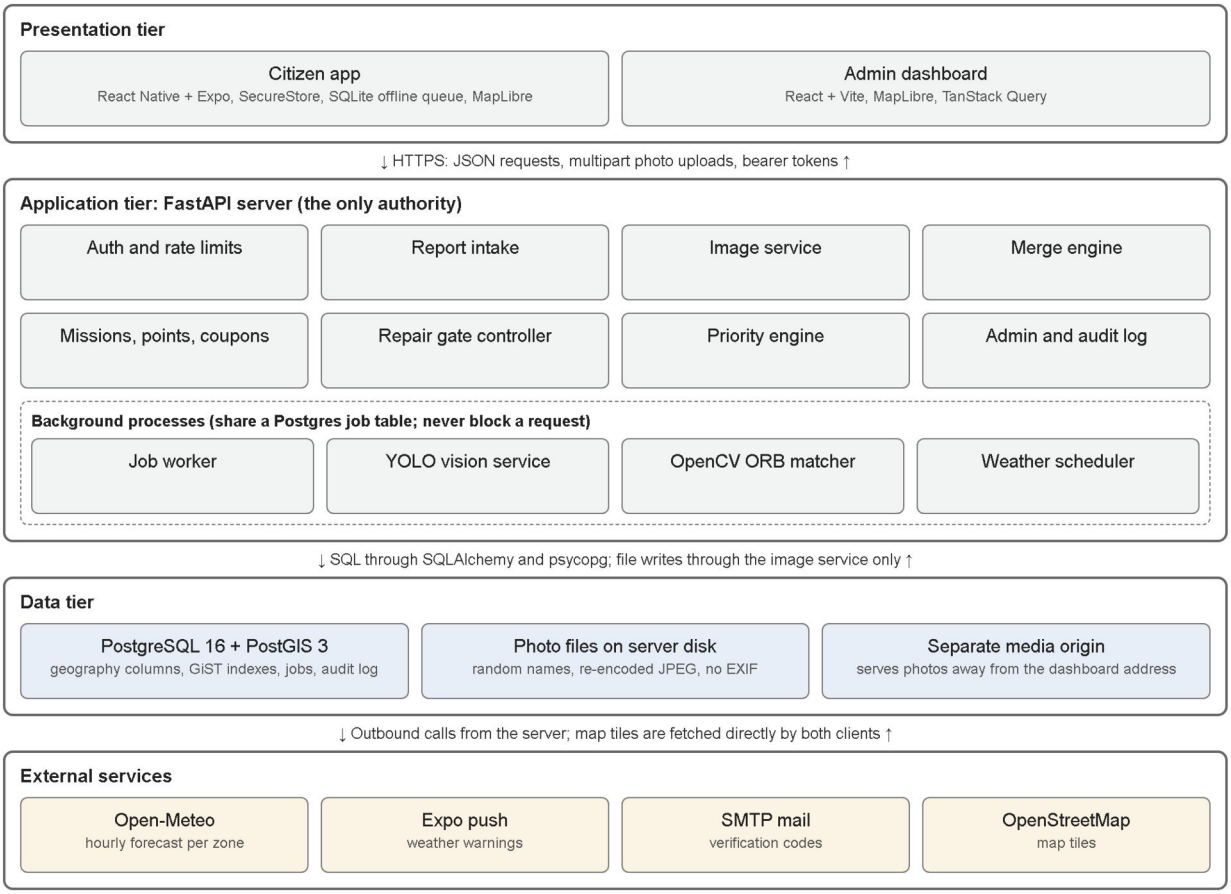


Figure 1: Detailed system architecture

Table 3: Principal architectural decisions

Decision	Rationale
Single FastAPI codebase	The detector is written in Python, so one language and deployment avoid an unnecessary network hop.
PostgreSQL job table in place of a message broker	Claiming with SELECT FOR UPDATE SKIP LOCKED is safe across workers and avoids an additional service.
Inline merging and background analysis	An indexed lookup is fast, whereas inference takes seconds.
Rate-limit counters in the database	Counts survive restarts and remain correct across server processes.
Photographs on disk, served from a second origin	One disk provides enough capacity, and a separate origin prevents a crafted file from executing in an administrator's session.
Inference on the server	Results remain consistent across handsets and application versions.

Table 4: API route groups

Route group	Representative routes	Access
Authentication	POST /auth/register, /auth/verify-email, /auth/resend-code, /auth/login, /auth/refresh	Public
Profile and device	GET, PATCH /me; POST /devices	User
Reporting	GET /categories; POST /reports; GET /reports/mine, /reports/{id}	User
Map	GET /pins, /pins/{id}	User
Weather	GET /weather/warnings; PUT /devices/me/zone; POST /notifications/{id}/received	User
Missions	GET /missions/available, /missions/{id}; POST /pins/{id}/confirm, /missions/{id}/accept, /missions/{id}/submit	User
Rewards	GET /points/me, /leaderboard, /coupons; POST /coupons/{id}/claim	User
Administration	GET /admin/queue, /admin/claims, /admin/audit, /admin/stats; POST /admin/pins/{id}/split, /admin/pins/{id}/status, /admin/claims/{id}/decision, /admin/coupons, /coupons/redeem; GET, PUT /admin/ranking-config	Admin

4. Domain Model

The domain model presents the principal real-world concepts and their relationships, independent of implementation. An account files reports, and each report describes one hazard pin. Missions direct people to pins. An approved repair claim closes the pin and awards points.

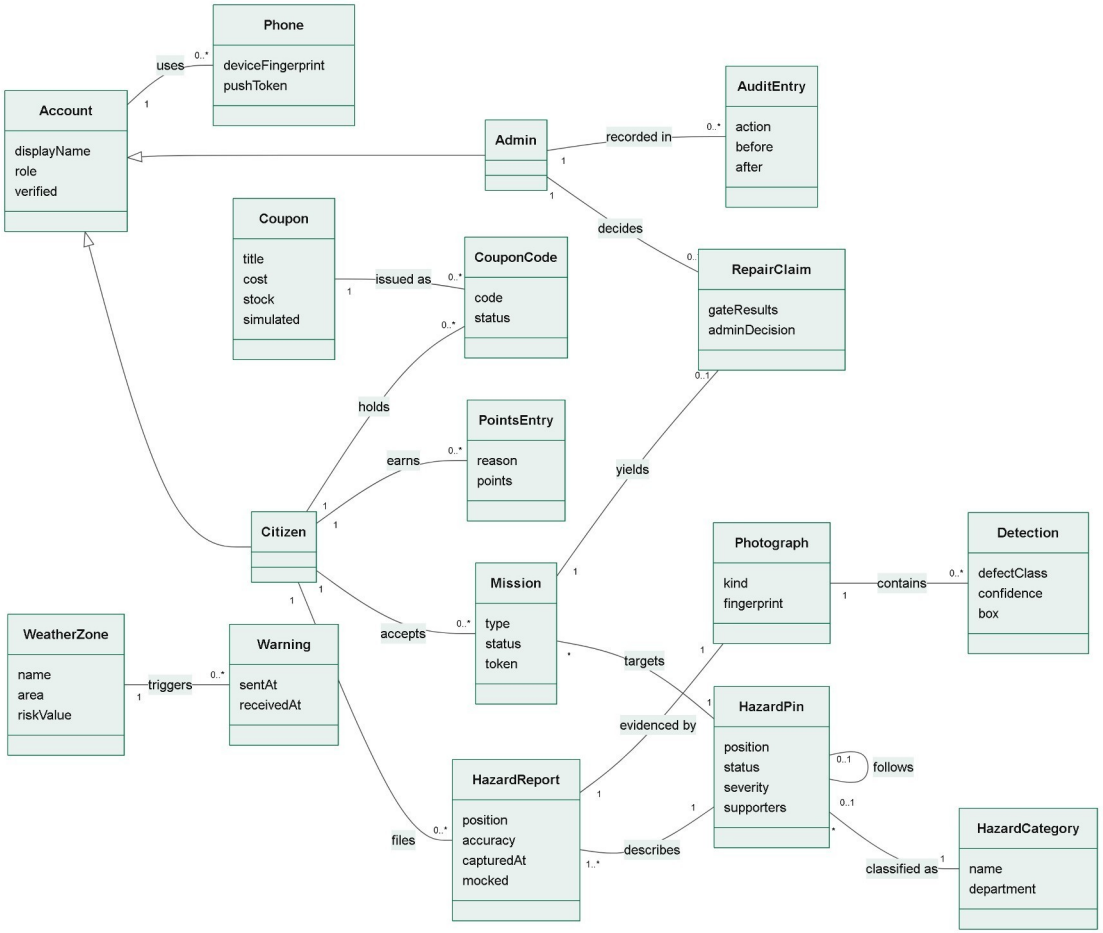


Figure 2: Domain model

5. Class Diagram

Entity classes map to database tables through SQLAlchemy, and service classes hold the business rules. A service may be written in Python as a module of functions with the same responsibilities. Figure 3 covers accounts, report intake, merging and analysis, and Figure 4 covers ranking, weather, repair checks and rewards.

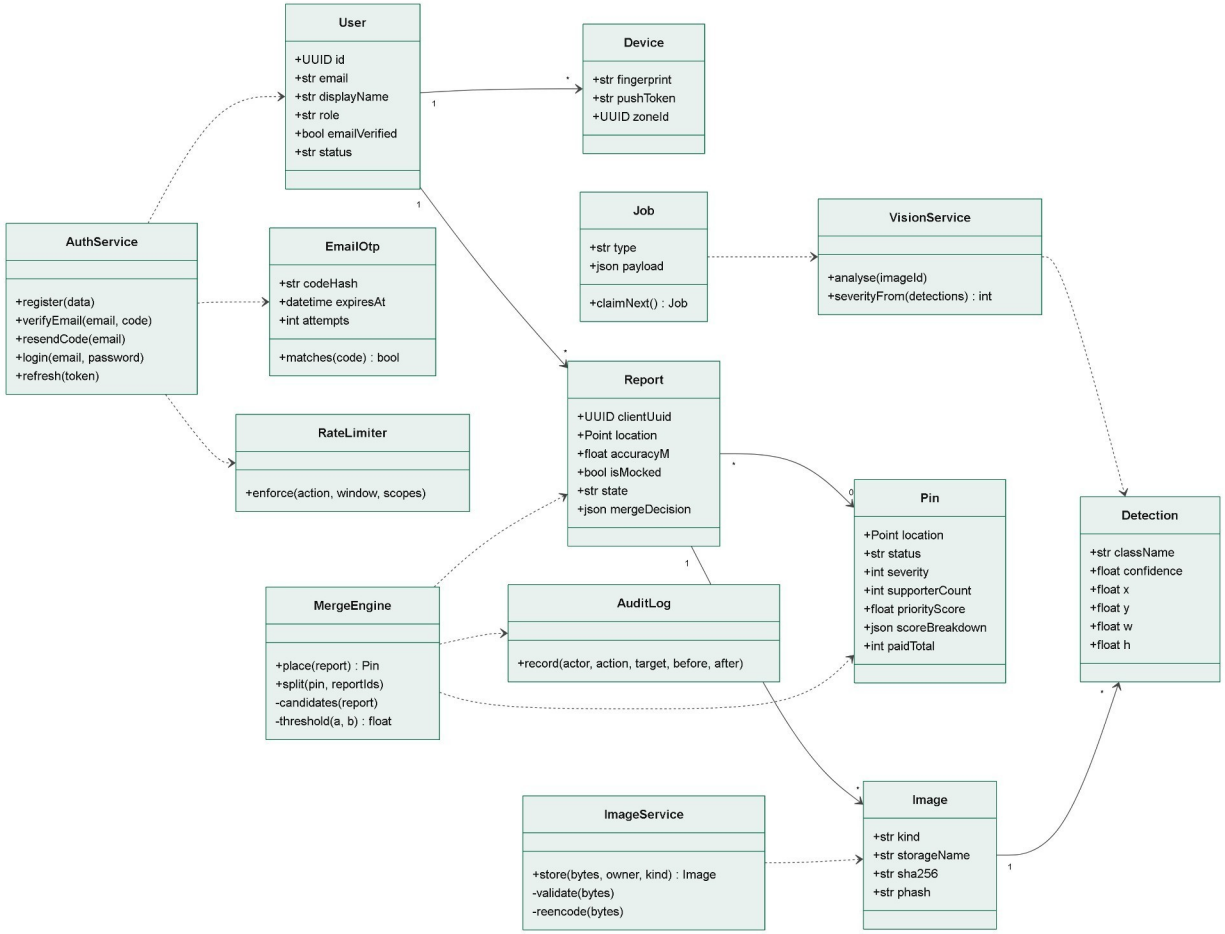


Figure 3: Class diagram, part 1: accounts, reports, merging and vision

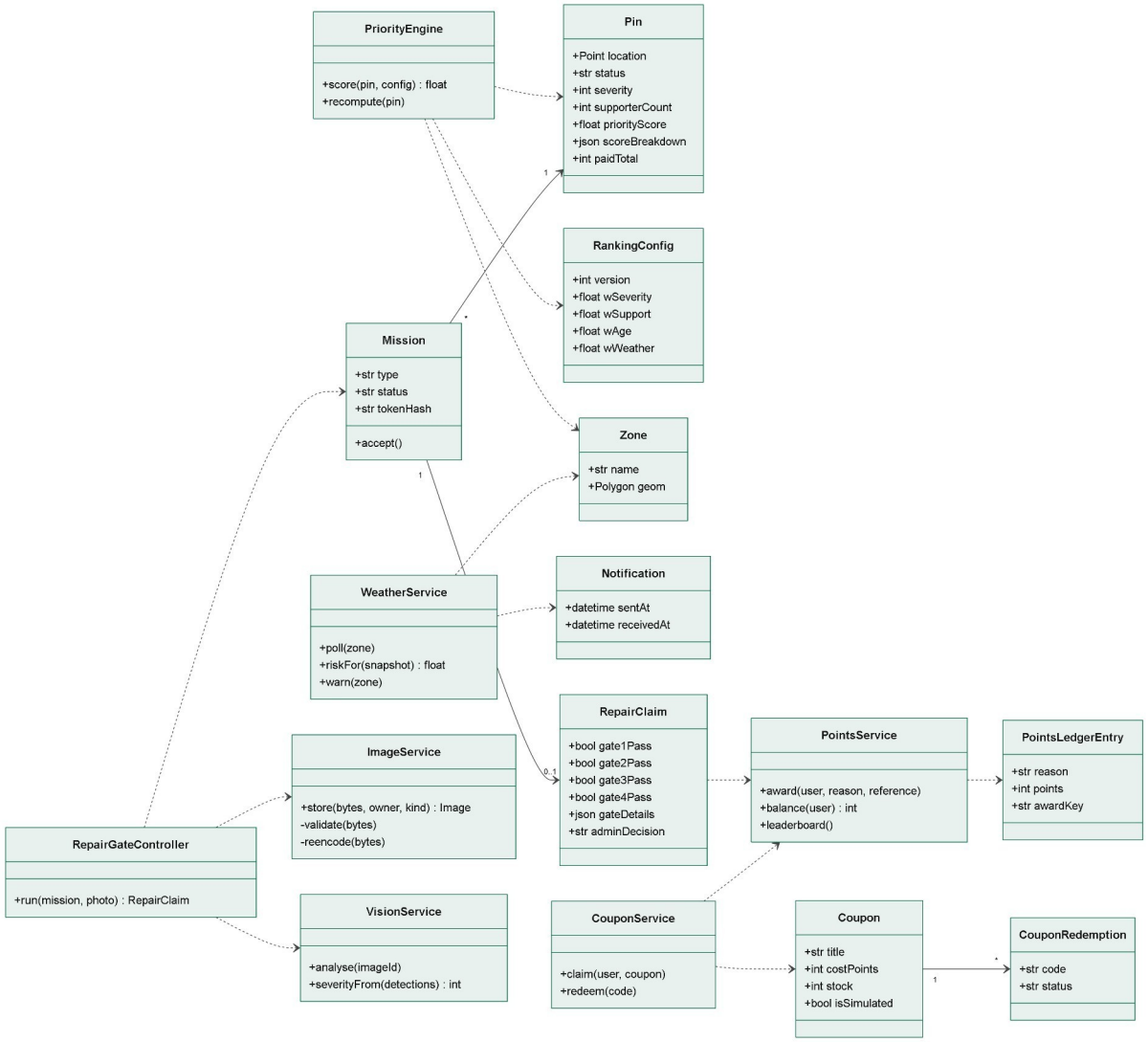


Figure 4: Class diagram, part 2: ranking, weather, repair checks and rewards

Table 5: Responsibilities of the service classes

Class	Principal responsibility
AuthService	Handles registration, code verification, login and token refresh.
RateLimiter	Increments a counter for each action and scope, and refuses requests above the limit.
ImageService	Is the only component permitted to write photograph files; it validates, re-encodes, names and hashes them.
MergeEngine	Acquires an advisory lock per category, finds candidates, applies the pair threshold (which uses reported GPS accuracy [6]), attaches or creates a pin, and records its decision.
VisionService	Loads the model trained on RDD2022 [5] and local data once, stores detections and computes severity.
PriorityEngine	Provides a pure scoring function and the triggers that recompute a pin's score.
WeatherService	Polls Open-Meteo [10], computes risk, suppresses repeat warnings and sends push messages.
RepairGateController	Runs the gates in order, applies the evaluation switches and stores each measurement. It also notes the mock-location flag reported by the phone [11].
PointsService	Awards points using the unique key, enforces the cap and builds the leaderboard.
CouponService	Claims coupons under a row lock and redeems them with one conditional update.
AuditLog	Provides a single helper that writes an audit record within the caller's transaction.

6. Database Diagram

The database uses PostgreSQL 16 with the PostGIS 3 extension. Identifiers are UUIDs and times are stored in UTC. The physical tables are grouped below by subject area. A table owned by another area appears only with its primary key, to show the relationship.

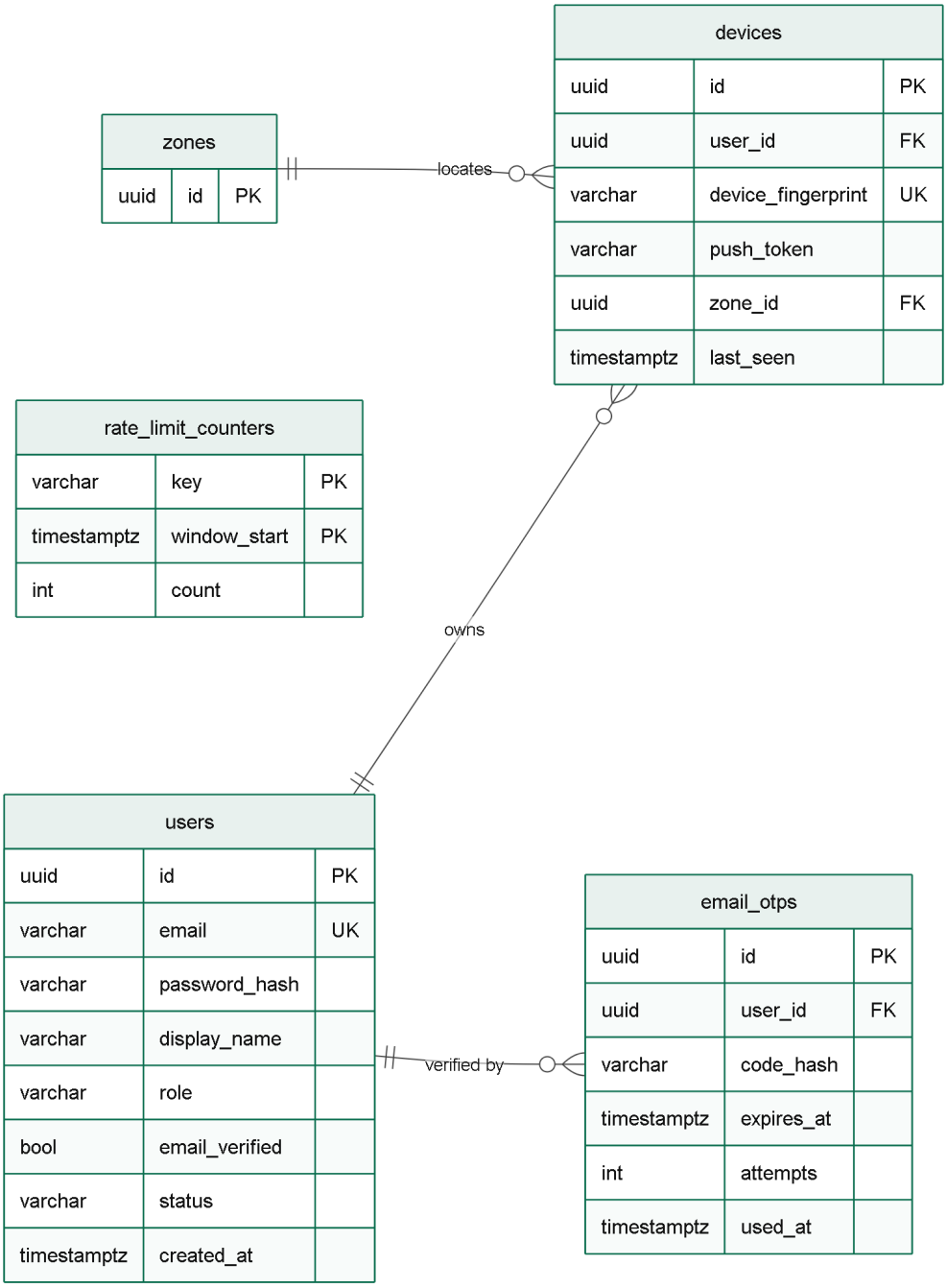


Figure 5: Database diagram: accounts and rate limiting

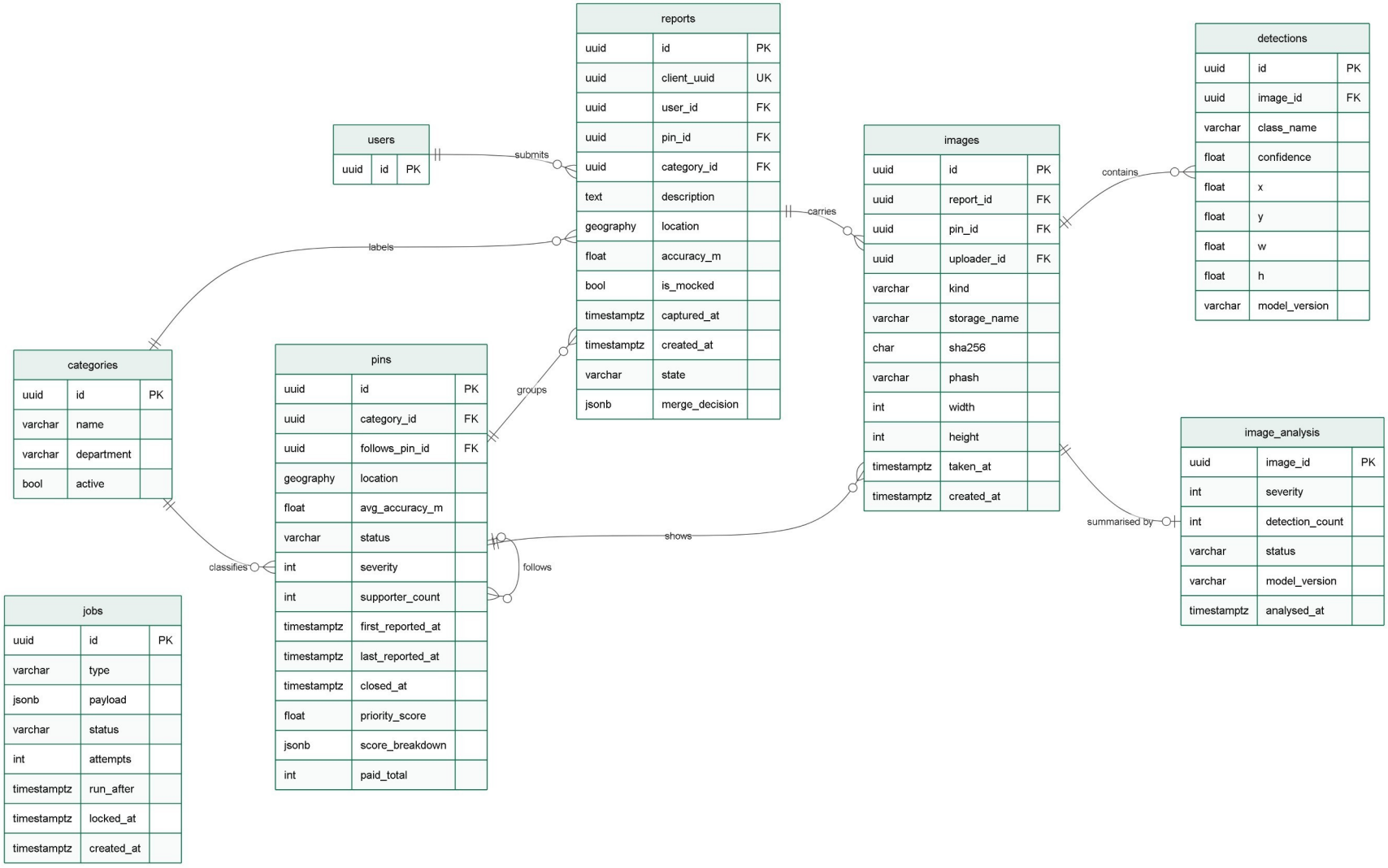


Figure 6: Database diagram: reports, pins, images and analysis

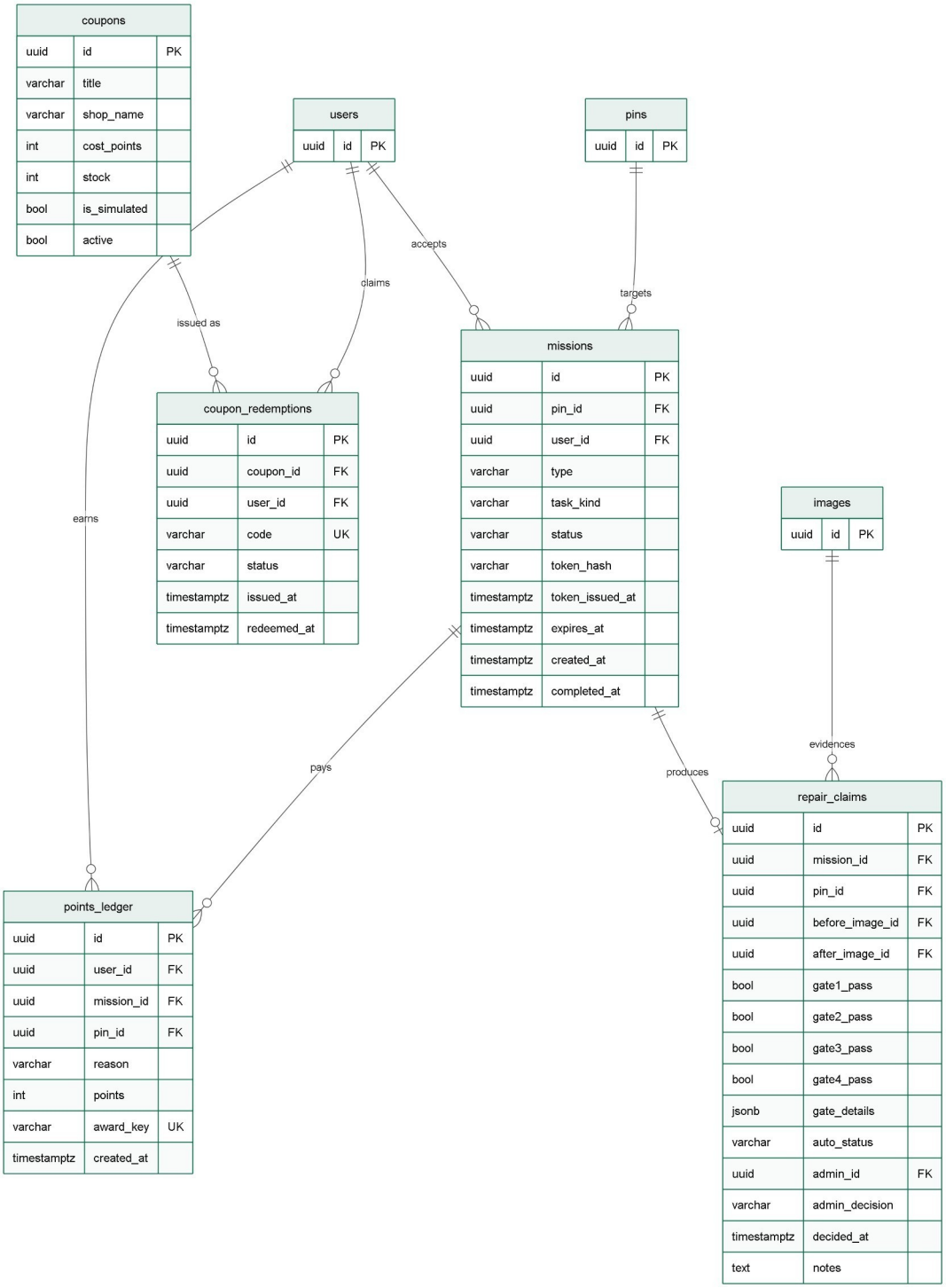


Figure 7: Database diagram: missions, repair claims, points and coupons

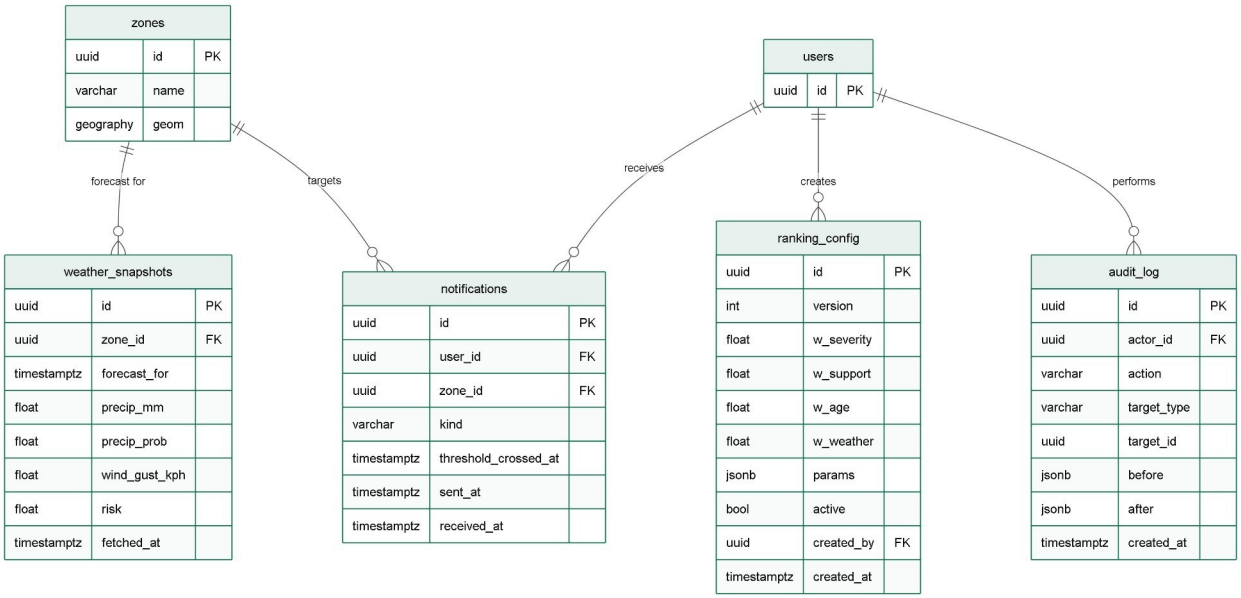


Figure 8: Database diagram: weather, ranking settings and audit log

Table 6: Principal constraints and indexes

Constraint or index	Purpose
GiST index on reports.location, pins.location, zones.geom	Supports indexed nearby-pin and viewport queries.
Composite index on pins (category_id, status, last_reported_at)	Narrows merge candidates efficiently.
Unique reports.client_uuid	Prevents a repeated offline upload from creating a second report.
Unique points_ledger.award_key	Makes a replayed award violate the constraint, so it is not paid twice.
Unique coupon_redemptions.code	Ensures that each issued code is distinct.
Unique devices.device_fingerprint	Ensures that a shared phone's push token belongs only to the current account.
Unique users.email; check constraints on role and status	Enforces one account per email and valid role and status values.

7. Entity Relationship Diagram (ERD)

The entity relationship diagrams show the same data at the conceptual level, without attributes. Figure 9 covers users, reports, pins and images, and Figure 10 covers missions, rewards and weather. A report belongs to a pin but remains a separate record, so a split can be reversed. The relationship from PIN to itself records that a new pin follows a closed pin, so failed repairs and recurring defects can be identified.

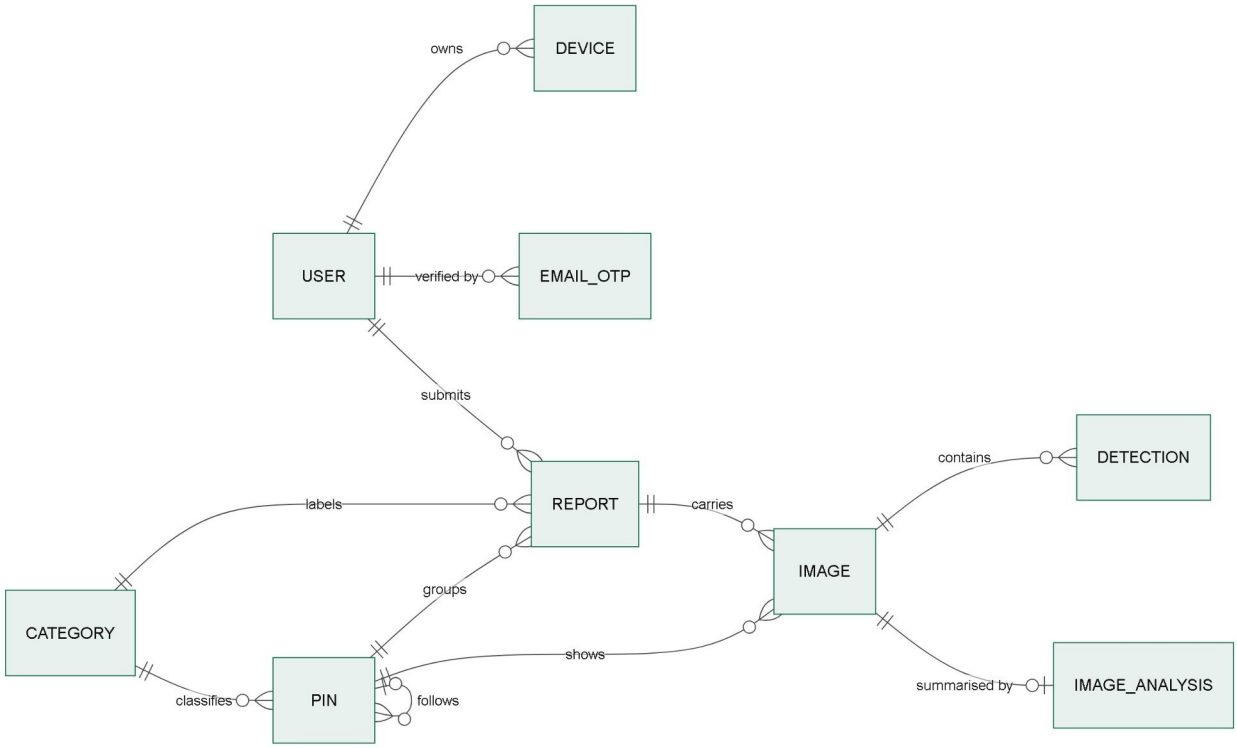


Figure 9: Entity relationship diagram: users, reports, pins and images

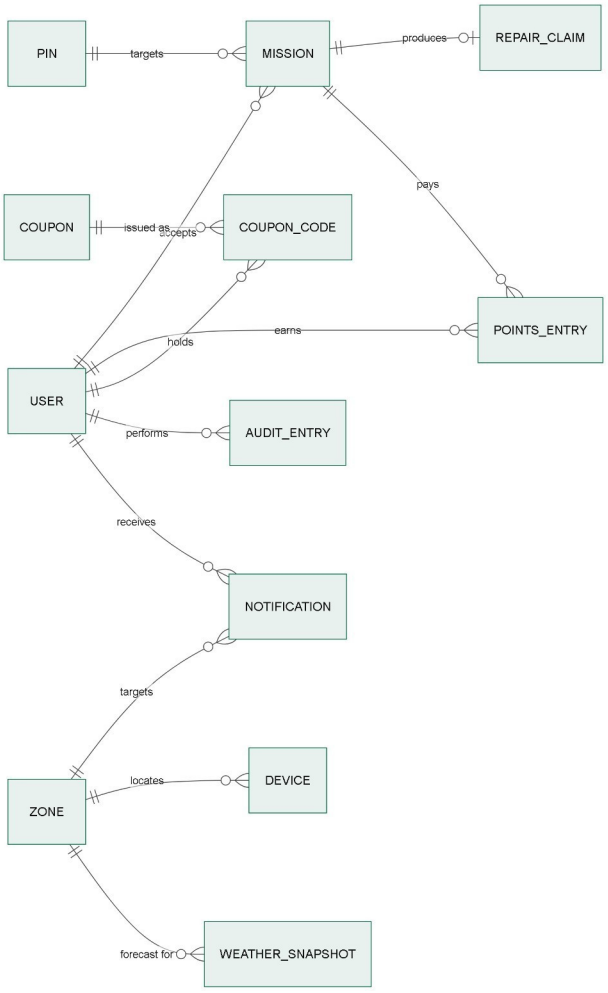


Figure 10: Entity relationship diagram: missions, rewards and weather

8. Sequence Diagram

Figures 11 to 13 show the three interactions with the greatest technical risk.

Registration and login. The verification email is sent after the transaction commits, so a mail failure cannot lose the account, and the user can request a new code.

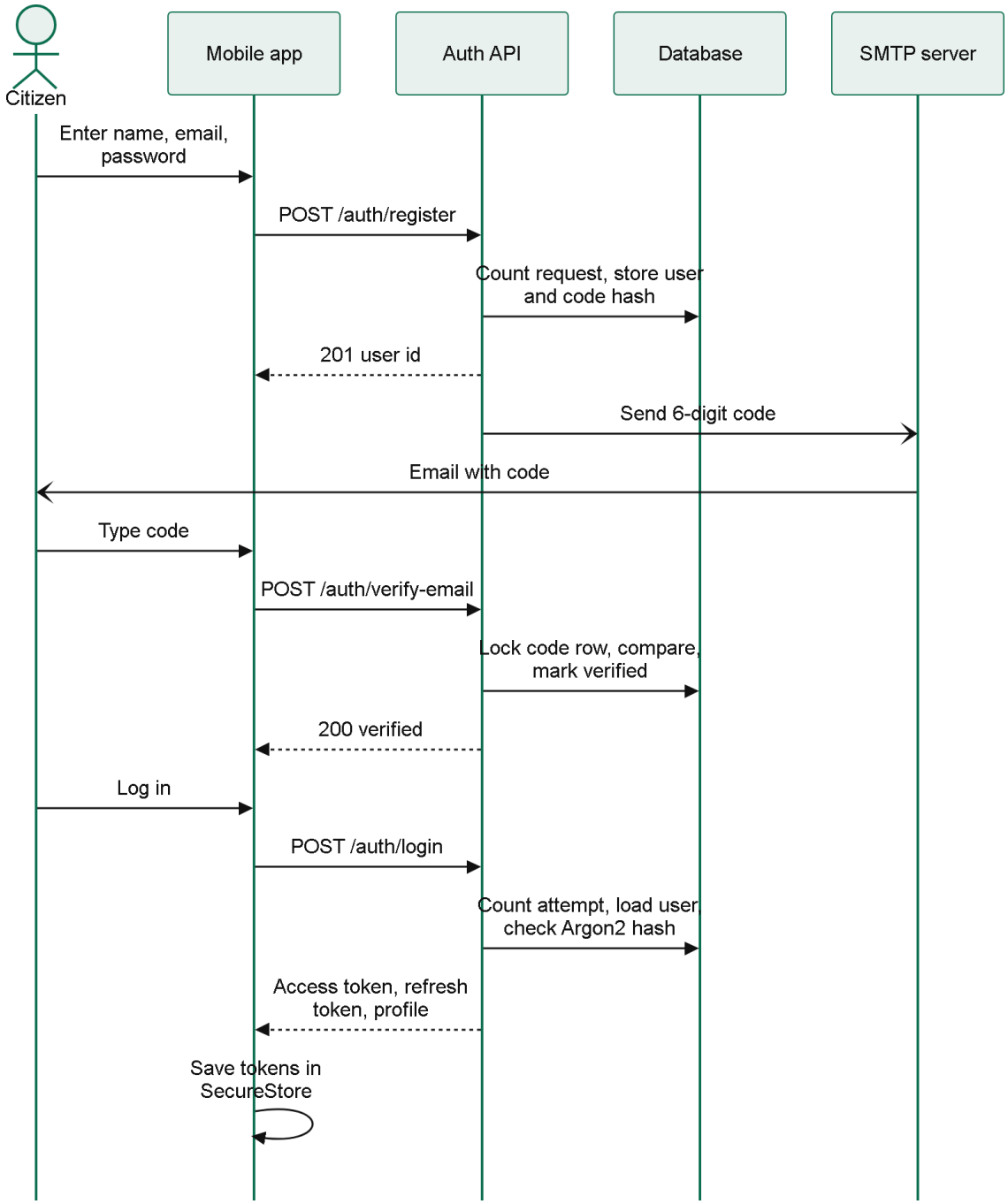


Figure 11: Sequence diagram: registration, verification and login

Report submission and merging. The client identifier makes retries safe. The merge runs in one transaction under a per-category lock, which prevents two simultaneous reports of one pothole from creating two pins.

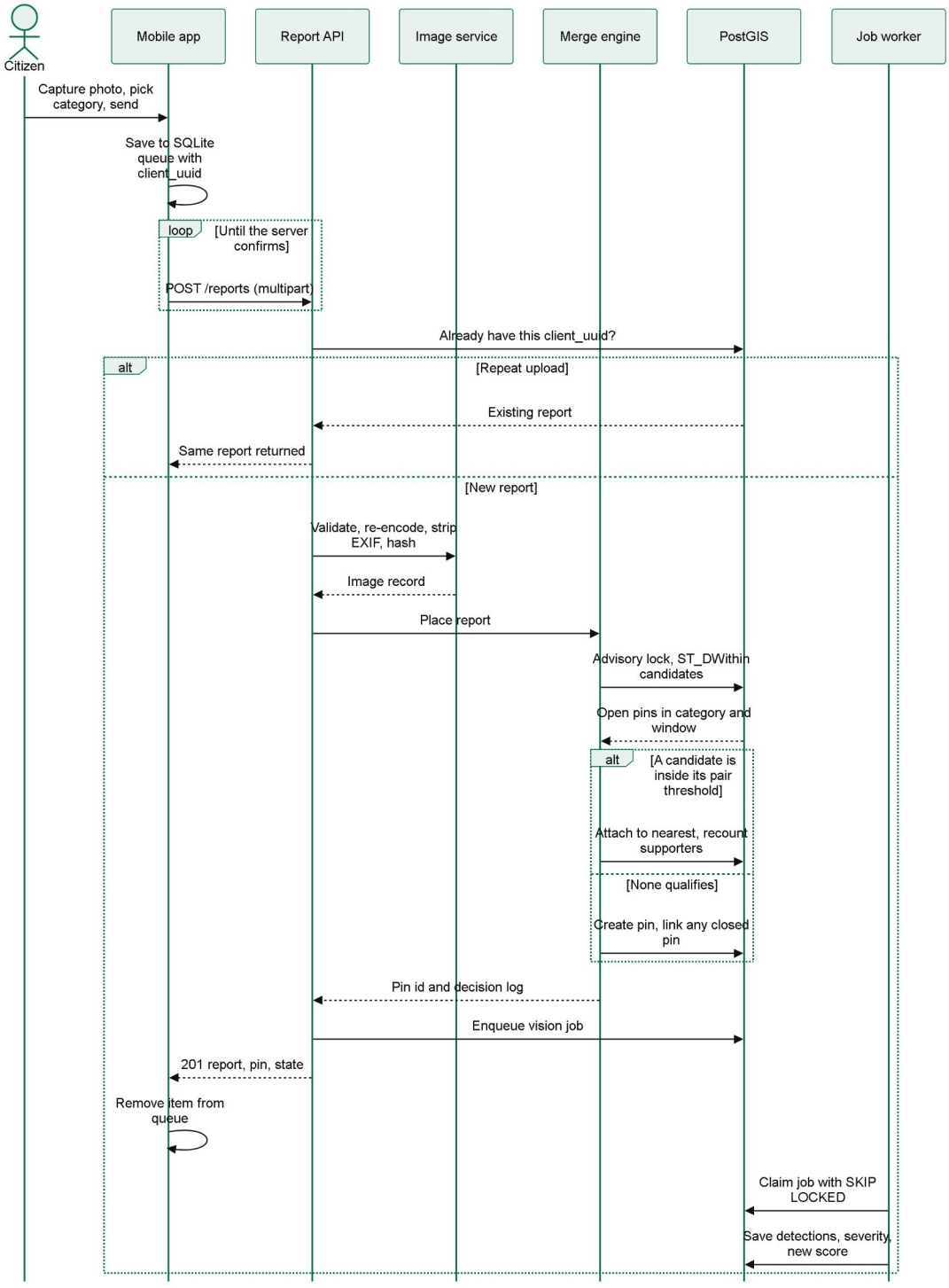


Figure 12: Sequence diagram: report submission and merging

Repair verification. Inexpensive gates run first, and processing stops at the first failure. Points are awarded only after the administrator's decision, and a unique key protects that step from repetition.

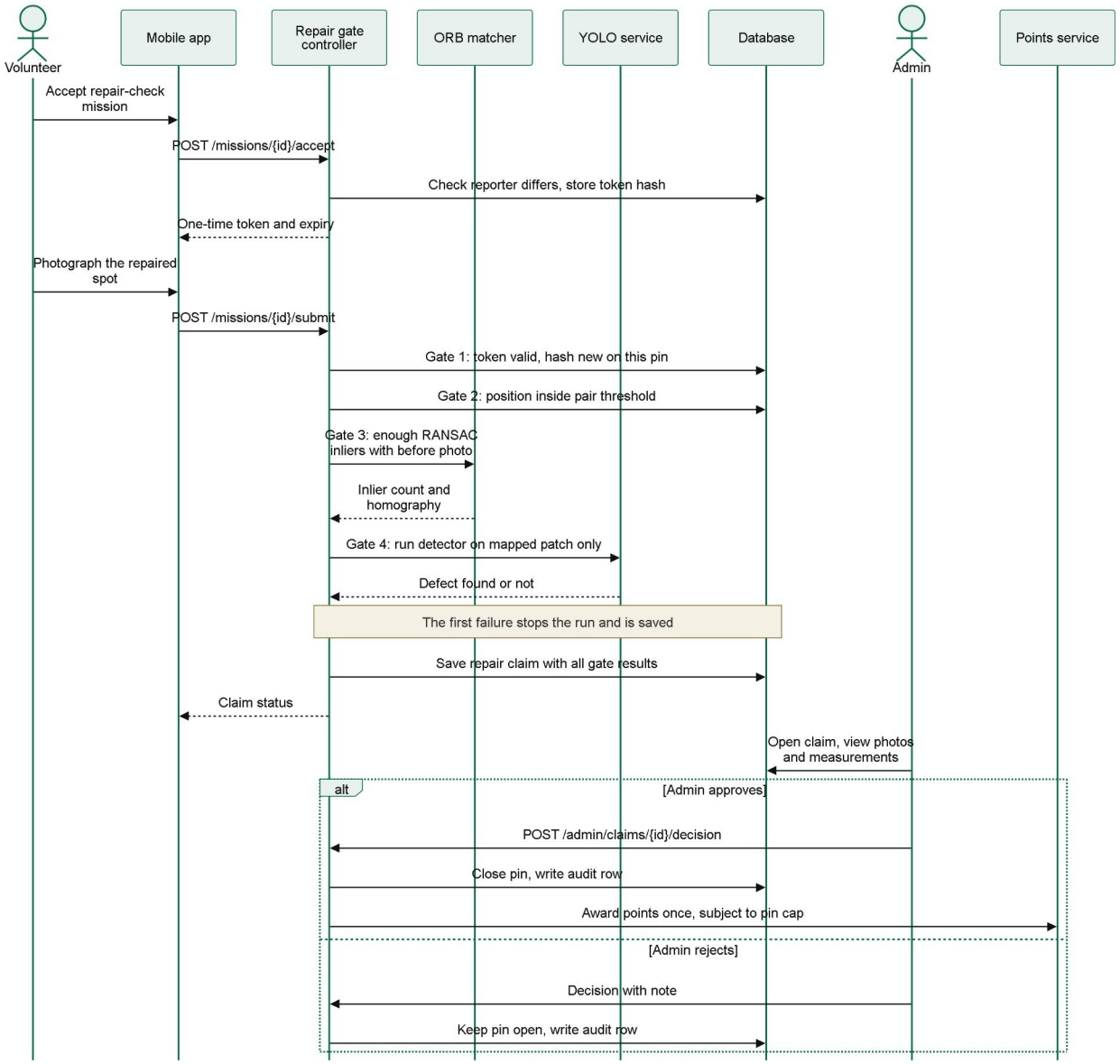


Figure 13: Sequence diagram: repair verification and approval

9. System Interface Design

The following screens are planned layouts and not images of completed software. Names and numbers in them are illustrative samples. The citizen application uses large controls and short workflows for outdoor use, and a bottom bar gives access to the five main areas.

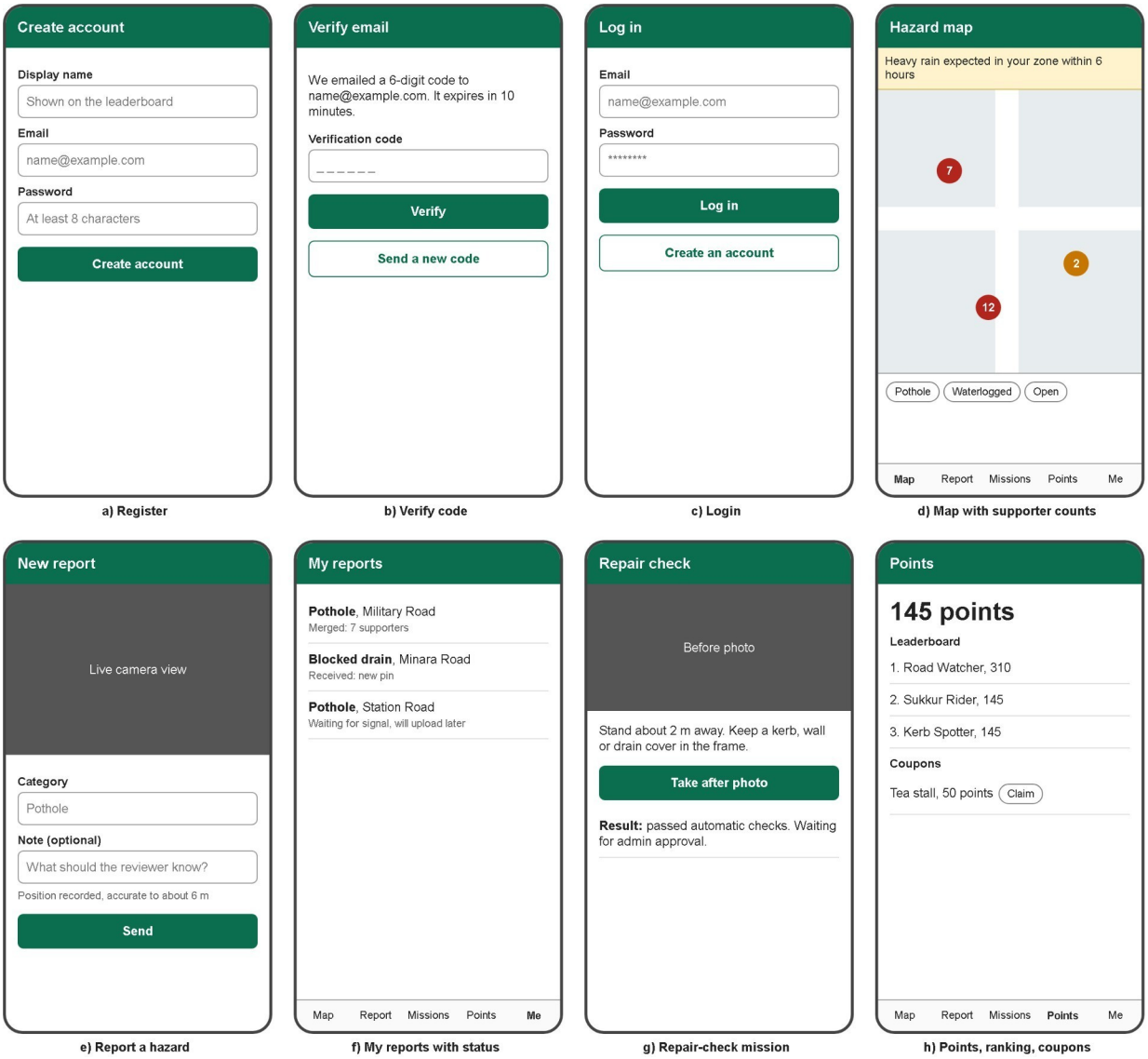
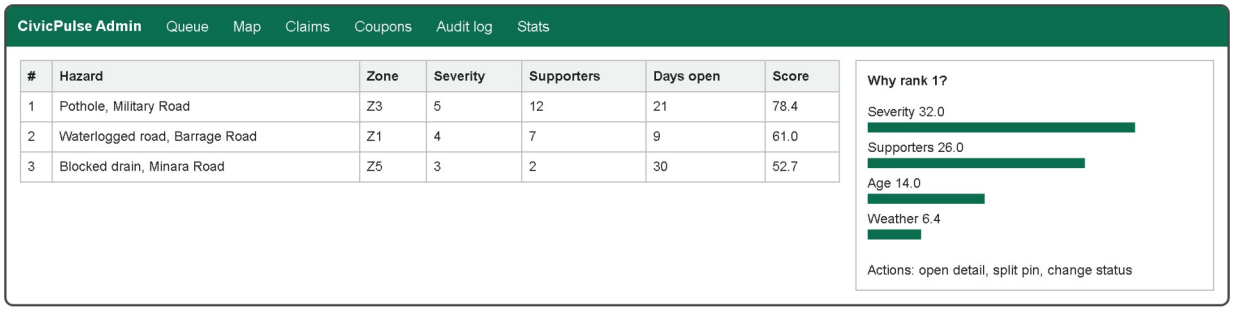


Figure 14: Citizen application screen designs

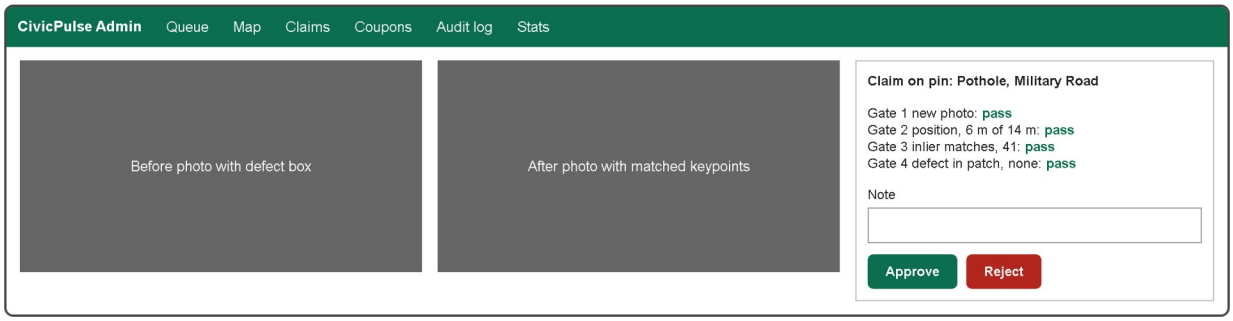
Table 7: Citizen application screens

Screen	Purpose	Use case
Register, Verify code, Login	Account creation and access	UC-01 to UC-03
Map	Viewing pins, supporter counts and weather warnings	UC-07, UC-15
New report, My reports	Submitting reports and following their status, including queued items	UC-05, UC-06
Repair check	Taking the after photograph with framing guidance	UC-10
Points	Viewing balance, leaderboard and coupons	UC-16, UC-17

The dashboard is designed for desk use. The queue explains each hazard's rank, the claim review displays evidence and gate measurements side by side, and the audit log records each administrative change. Stored text is rendered through React's default escaping.



a) Ranked queue with score breakdown (sample values for illustration)



b) Gate 5 claim review (sample values for illustration)



c) Audit log viewer (sample values for illustration)

Figure 15: Administrator dashboard screen designs

10. Test Cases

The following checks are planned, and results will be recorded as each module is completed. Unit, database and API tests run under pytest against a real PostGIS container, and mobile tests are performed manually on an Android phone.

Table 8: Planned test cases

ID	Scenario	Expected result	Type
TC-01	Register an existing email in a different letter case	Status 409; no second account	API
TC-02	Submit five incorrect codes, then the correct one	Code locks on the fifth attempt; correct code receives 423	API
TC-03	Submit a code after 10 minutes	Refused as expired	API
TC-04	Attempt login before verification	Status 403 email_not_verified	API
TC-05	Make six login attempts in one minute	Sixth attempt receives 429 with Retry-After	API
TC-06	Use a citizen token on an administrator route	Status 403	API
TC-07	Use a refresh token as an access token, and the reverse	Status 401 in both cases	API
TC-08	Log in, then restart the application with no signal	User remains logged in	Mobile
TC-09	Report in airplane mode, then reconnect	Report reaches the server exactly once	Mobile
TC-10	Send the same client_uuid twice	One report is stored	API
TC-11	Upload a script renamed with a .jpg extension	Rejected	Security
TC-12	Upload a photograph containing GPS EXIF data	Stored copy contains no EXIF data	Security
TC-13	Upload a 16 MB file	Rejected	Security
TC-14	Supply a file name containing ../	Ignored; the server assigns the name	Security
TC-15	One account reports one hazard five times	Supporter count remains 1	Database
TC-16	Five accounts report one hazard	Supporter count is 5	Database
TC-17	Report a different category at the same location	New pin	Database
TC-18	Report at the location of a closed pin	New pin linked to the closed pin	Database
TC-19	Report 90 days after the last report on a pin	New pin	Database
TC-20	Submit opposite-kerb reports 12 m apart, and reports of two defects 10 m apart	Outcome recorded and discussed	Experiment
TC-21	Split a pin	Two pins, all reports intact, audit record written	API
TC-22	Sweep the merge radius from 5 m to 40 m on the labelled set	Precision and recall curve produced	Experiment
TC-23	Upload a photograph with no defect	Marked unanalysed; category default used; no failure	Unit
TC-24	Evaluate public-only and fine-tuned models on held-out Sukkur photographs	Two mAP@0.5 values reported	Experiment
TC-25	Compute severity and score for hand-worked inputs	Values match the manual calculation	Unit
TC-26	Add a supporter to a pin	Score does not decrease	Unit
TC-27	Score one pin in a high-risk zone and in a calm zone	High-risk version ranks higher	Unit
TC-28	Compare the score with rankings of about 50 hazards by three people	Spearman's rho and largest disagreements documented	Experiment
TC-29	Repeatedly trigger a mocked threshold crossing	Push arrives in the zone only; median delay and targeting recorded	Experiment
TC-30	Force the forecast request to fail	Scheduler continues using the last snapshot	Unit

ID	Scenario	Expected result	Type
TC-31	Submit the original before photograph as the after photograph	Rejected at Gate 1	API
TC-32	Submit a photograph of clean road elsewhere	Rejected at Gate 2	Experiment
TC-33	Submit a different, genuinely repaired pothole nearby	Rejected at Gate 3	Experiment
TC-34	Frame the defect out of shot, or photograph from far away	Rejected at Gate 4	Experiment
TC-35	Run the faked set with Gates 2 to 4 disabled, then enabled	Fakes pass when disabled; false-accept and false-reject rates recorded when enabled	Experiment
TC-36	The reporter of a pin attempts its repair check	Refused	API
TC-37	Submit a claim that passes Gates 1 to 4, with no administrator decision	No points in the ledger	API
TC-38	Award each mission type	Each amount equals the published table	Unit
TC-39	Replay an award request	Paid once	Database
TC-40	Attempt payment above the per-pin cap	No further payout	Database
TC-41	Read a leaderboard with tied totals repeatedly	Order is unchanged	Database
TC-42	Redeem one code from two connections simultaneously	One success and one status 409	Database
TC-43	Store a description containing a script tag, then view it	Displayed as plain text	Security
TC-44	Perform each administrator action	One audit record for each action	API
TC-45	Request the public pin list	No reporter identity in the response	API

11. References

[1] Provincial Disaster Management Authority, Government of Sindh, Flood 2022 in Sindh, from June 20, 2022 to March 31, 2023, p. 17. [Online]. Available: https://pdma.gos.pk/Documents/Reports/Flood%202022%20In%20Sindh.pdf (accessed Sep. 3, 2026).

[2] "Murad allocates Rs1.5bn for repair of few select rain-damaged roads in city," Dawn. [Online]. Available: https://www.dawn.com/news/1859688

[3] "Govt celebrates Citizen Portal success despite only 30% satisfaction rate," ProPakistani, Aug. 6, 2026. [Online]. Available: https://propakistani.pk/2026/08/06/govt-celebrates-citizen-portal-success-despite-only-30-satisfaction-rate/

[4] mySociety, "FixMyStreet in the UK." [Online]. Available: https://www.mysociety.org/community/fixmystreet-in-the-uk/

[5] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2022: A multi-national image dataset for automatic road damage detection," Geoscience Data Journal, 2024. [Online]. Available: https://arxiv.org/abs/2209.08538

[6] National Coordination Office for Space-Based Positioning, Navigation, and Timing, "GPS Accuracy." [Online]. Available: https://www.gps.gov/gps-accuracy (accessed Sep. 4, 2026).

[7] PostGIS Project, "ST_ClusterDBSCAN," PostGIS Reference. [Online]. Available: https://postgis.net/docs/ST_ClusterDBSCAN.html (accessed Sep. 4, 2026).

[8] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2011, pp. 2564-2571, doi: 10.1109/ICCV.2011.6126544.

[9] Ultralytics, "Ultralytics YOLO Docs." [Online]. Available: https://docs.ultralytics.com/ (accessed Sep. 4, 2026).

[10] Open-Meteo, "Weather API Features." [Online]. Available: https://open-meteo.com/en/features (accessed Sep. 4, 2026).

[11] Expo, "Location: LocationObject," Expo SDK Documentation. [Online]. Available: https://docs.expo.dev/versions/latest/sdk/location/ (accessed Sep. 4, 2026).

[12] A. Kumar, Sarichand, and R. Bai, "CivicPulse: A civic platform that proves a road hazard was actually repaired," Final Year Project Proposal 23F-90, Dept. of Computer Science, Sukkur IBA University, 2026.